

AWS GenAI 課程 · 第 4 單元

程式開發人員的日常

盧建霖 • 2026-09-19

站會開了一年，還是不知道彼此在做什麼？
今天把工程師的一週，變成看得到的產物。

Agenda

- 01 儀式與票：一個 Sprint 裡裝什麼
- 02 系統化 Debug
- 03 跨職能協作與角色轉變
- 04 AI 的三個切點與零程式碼實作

TODAY'S GOAL

拿到一段模糊需求，能補成可以開工的規格，再用三句話講清楚進度與阻礙。

前三段講工程師的一週長什麼樣，最後一段動手做，不寫一行程式。

先定義名詞，後面才好講

這六個詞今天會反覆出現，第一次出現就先給名字

敏捷 **agile**

用短週期、常回饋來應付需求會變的開發方式，**Scrum** 是最常見的一種做法。

Sprint

一個月或更短的固定時段，所有工作都在裡面發生。

票 **ticket**

一件要做的事的紀錄，**GitHub Issue** 就是一種票。

驗收條件 **acceptance criteria**

一張票怎樣算做完的可檢查清單。

追蹤 ID **trace ID**

請求進系統時發的識別碼，跟著它穿過所有服務。

跨職能 **cross-functional**

一個團隊裡就有把事情做完需要的全部技能。

每個儀式、每張票，都在解決一個具體問題

- 一、儀式是回饋點，忘了它解決的問題就走鐘
- 二、站會的判準：會後有沒有人的計畫改變
- 三、沒寫的例外情境，以後都會變成 **bug**

Sprint 是容器，四個事件在裡面

Scrum 是處理複雜問題的輕量框架；Sprint 是它固定長度的一段時間

Scrum 指南 2020：Sprint 是容器，四個事件都在裡面



1 一個 Sprint 一個月或更短，前一個結束，下一個緊接著開始

2 四個事件按順序：Planning、Daily、Review、Retrospective

3 Product Backlog（產品要做的事的排序清單）的精煉持續進行

五個儀式，各解決一個問題

Sprint Goal 是這個 Sprint 要達成的一個目標，Planning 結束前要定下來

儀式	解決什麼問題	常見的走鐘
Sprint Planning	這兩週要做什麼、為什麼	只排工作，沒講為什麼
Daily Scrum	今天怎麼往 Sprint Goal 前進	變成向主管報進度
Refinement	下一批工作夠不夠清楚	沒做，Planning 當場才問需求
Sprint Review	做出來的東西對不對	變成成果發表會
Retrospective	下次怎麼做得更好	變成抱怨大會或跳過

忘了儀式在解決的問題，它就變成例行公事。

站會走鐘，就變成進度報告

Daily Scrum 是 Developers（團隊裡實際做出東西的人）的 15 分鐘事件

比較	進度報告式站會	指南要的站會
對誰講什麼	對主管講昨天做 A、今天做 B	對團隊講離 Sprint Goal 還差什麼
講完之後	每個人講完就散	有人的今天計畫因此改變
時間	15 分鐘變 40 分鐘	15 分鐘，細節會後小組談

判準只有一個：會後有沒有人的計畫因為這場會而改變。

儀式太多，砍掉就敏捷了？

× 大家通常會想

儀式太多，砍掉幾個，團隊就敏捷了。

砍掉的是回饋點，站會變成進度報告也一樣失去意義。

✓ 實際上

每個儀式都是一個「早一點發現不對」的機會。

跟 CI (Continuous Integration，持續整合) 那一課的邏輯一模一樣。

QUIZ 01

一場站會結束後，沒有任何人的今天計畫改變。這場會最接近什麼？

A 成功的 Daily Scrum

B 進度報告

C Retrospective

D Refinement

想 30 秒，等一下翻下一頁對答案。

QUIZ 01 · ANSWER

一場站會結束後，沒有任何人的今天計畫改變。這場會最接近什麼？

A 成功的 Daily Scrum

B 進度報告

C Retrospective

D Refinement

B。判準只有一個：會後有沒有人的計畫因為這場會而改變；沒有，這場會就是進度報告（講師的說法）。

規格在**第三格**就定了

一張票從 PRD (Product Requirements Document, 產品需求文件) 走到關閉, 共七格

票的一生：規格在第三格就定了，後面都在對照它



「規格不清」的漏洞就在第三格：驗收條件沒寫，後面每一格都要回頭問

- 1 七格：需求文件、拆票、寫驗收條件、進 Sprint、進行中、驗收、關閉
- 2 第三格上色：驗收條件定了，後面每一格都在對照它
- 3 驗收條件沒寫，後面每一格都要回頭問

從一段話到一張票，分四層

使用者故事（user story）的標準格式：As a [user], I want [goal] so that [reason]

層	長什麼樣	例子
PRD 的一段話	模糊、面向使用者	「使用者要能匯出報表」
使用者故事	身為誰、我想要什麼、為了什麼	身為店長，我想匯出當月訂單 CSV，為了對帳
驗收條件	可檢查的句子，一條一個情境	Given、When、Then（下一頁）
拆成 ticket	一張一件事，切到一天內做得完	匯出按鈕、CSV 產生、空資料時的提示

加上驗收條件，故事才可測試；「切到一天內」是講師的做法。

驗收條件用 **Given**、**When**、**Then** 寫

這三個字是 Gherkin 語言的關鍵字，用來寫可執行的規格；借它的格式就夠了

關鍵字	寫什麼	例子
Given 前提	系統在什麼狀態	店長已登入，當月有 120 筆訂單
When 動作	使用者做了什麼	點「匯出 CSV」
Then 結果	應該看到什麼	下載 120 列的 CSV：訂單號、日期、金額
例外情境	邊界與錯誤	當月 0 筆：顯示「本月無訂單」，不產生檔案

一條一個情境，**例外也要寫**。

驗收條件跟著票走

一張票做完的判準：它名下的條件全過

1 每張票只認領它負責的那幾條

「匯出按鈕」那張票：When 點按鈕，Then 開始產生檔案。

2 例外情境也有票認領

「空資料時的提示」那張票，條件就是那條例外。

3 完成之定義（Definition of Done）管整個團隊

它是整個團隊對「做完」的正式描述，出自 Scrum 指南。

規格不清，先做再改比較快？

× 大家通常會想

規格不清就先做，做出來再改。

「再改」的成本是重做，而且會在 Review 時才發現。

✓ 實際上

規格不清是工程師最大的時間漏洞。

一條沒寫的例外情境，就是一個以後半夜被叫起來的 bug。

QUIZ 02

下列哪一條是合格的驗收條件？

- ☐ A 匯出功能要好用
- ☐ B Given 當月 0 筆訂單，When 點匯出，Then 顯示「本月無訂單」且不產生檔案
- ☐ C 匯出要快
- ☐ D 匯出 CSV

想 30 秒，等一下翻下一頁對答案。

QUIZ 02 · ANSWER

下列哪一條是合格的驗收條件？

A

匯出功能要好用

B

Given 當月 0 筆訂單，When 點匯出，Then 顯示「本月無訂單」且不產生檔案

C

匯出要快

D

匯出 CSV

B。A、C 不可檢查；D 沒有情境與結果。

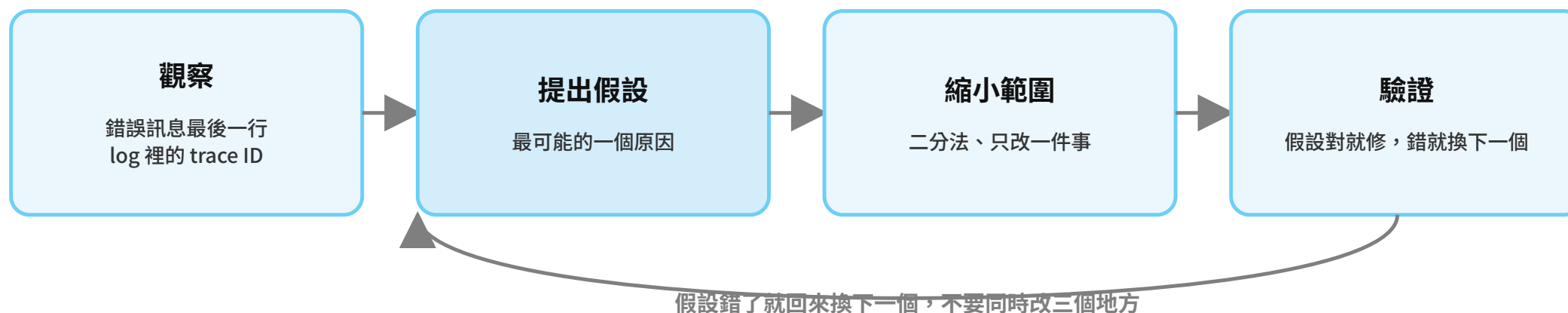
Debug 是在證明假設

- 一、先猜一個，一次只驗一個假設
- 二、**log** 分級與追蹤 **ID**，讓機器幫你篩
- 三、三個線索指同一邊，才下結論

先猜一個，再證明它對或錯

stacktrace 是程式出錯時印出的呼叫路徑；拿到它，先走這個循環

系統化 Debug：先猜一個，再證明它對或錯



1 四個框是一輪：觀察、提出假設、縮小範圍、驗證

2 觀察就是讀最後一行；縮小範圍就是二分法與最小重現

3 第二個框上色：新手最常跳過它，直接亂改

多改幾個地方，總有一個中？

× 大家通常會想

多改幾個地方，總有一個中。

同時改三個地方，好了也不知道是哪個，下次還會再犯。

✓ 實際上

一次只驗一個假設，每一步只改一件事。

假設錯了就換下一個；30 分鐘到了就停，把循環的紀錄寫成提問。

Log 分五級，讓機器幫你篩

log 是程式跑的時候留下的一行行紀錄；分級是每一行標一個嚴重程度

等級	用途	正式環境通常開到
DEBUG	診斷用的很細的輸出	關
INFO	正常運作中發生的事，狀態監控用	開
WARNING	有事不對勁但程式還在跑	開
ERROR	某個功能失敗了	開，通常會告警
CRITICAL	程式可能撐不下去	開，一定告警

查問題時只看某個等級以上。

追蹤 ID：一個請求，搜一次全串起來

OpenTelemetry 是一套開放標準，規定 log、指標、追蹤怎麼產生與傳遞

1 沒有追蹤 ID：靠時間戳猜

三個服務各自的 log，只能說「大概是下午三點那次」。

2 有追蹤 ID：每一行 log 都帶同一個 ID

例如 `trace_id=4bf9...`，搜一次，三個服務的 log 全部串起來。

3 OpenTelemetry 的講法：trace 由好幾段 span 組成

trace 是一個請求的完整路徑，span 是路徑上的一段；追蹤 ID 放在 span context 裡。

自己的錯還是環境的錯，看三個線索

拿到一段 stacktrace，在裡面找這三樣

看哪裡	偏自己的錯	偏環境的錯
第一個自己的檔案	有，錯在那一行附近	全部是套件或系統的檔案
最後一行的型別	<code>TypeError</code> 、 <code>KeyError</code> 這類邏輯錯	<code>ConnectionError</code> 、 <code>Timeout</code> 、 <code>Permission</code>
換一台機器或環境	一樣錯	不一樣

三個線索都指同一邊才下結論；只有一個就繼續驗。

QUIZ 03

stacktrace 全是套件與系統檔案，最後一行是 **ConnectionError**，換一台機器就好了。偏哪一邊？

A

自己的錯

B

環境的錯

C

無法判斷

D

Python 的錯

想 30 秒，等一下翻下一頁對答案。

QUIZ 03 · ANSWER

stacktrace 全是套件與系統檔案，最後一行是 **ConnectionError**，換一台機器就好了。偏哪一邊？

A 自己的錯

B 環境的錯

C 無法判斷

D Python 的錯

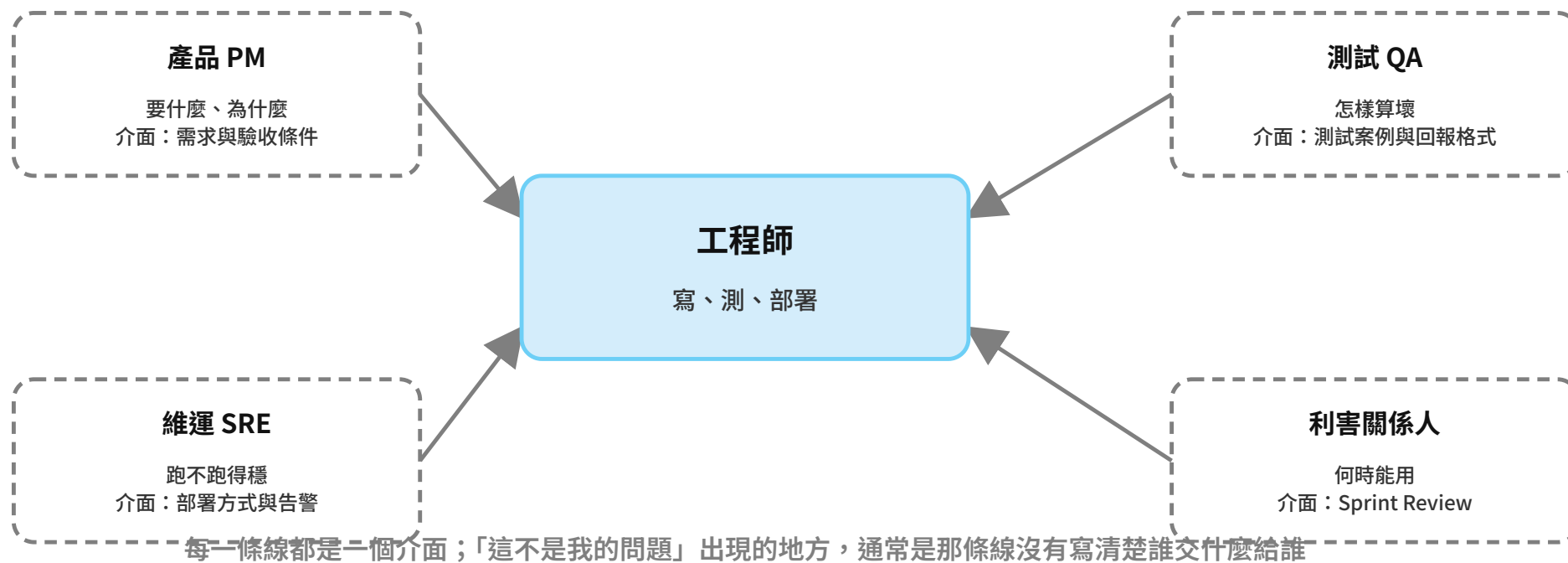
B。三個線索都指環境：沒有自己的檔案、型別是 **ConnectionError**、換一台機器結果不一樣。

介面要寫成產物，不要靠默契

- 一、摩擦多半來自沒設計好的介面
- 二、每種摩擦都能回推到一份沒寫的文件
- 三、**AI** 接手了寫的一部分，沒接手負責的那一部分

工程師有四條介面

PM 是 product manager，QA 是 quality assurance，SRE 是 site reliability engineer



- 1 PM 決定做什麼、為什麼；QA 定義與驗證「怎樣算壞」
- 2 SRE 負責系統跑得穩；利害關係人在意結果，但不在團隊裡
- 3 「這不是我的問題」出現的地方，通常是那條線沒寫清楚誰交什麼

每條介面，交一樣看得到的東西

對象	他們要的	你要的	介面上該有的東西
產品 PM	功能如期、符合需求	清楚的規格與優先順序	使用者故事、驗收條件、排序過的 backlog
測試 QA	可測的東西、穩定的測試環境	早一點知道「怎樣算壞」	測試案例、回報格式
維運 SRE	可部署、可觀測、出事有人接	環境穩定、部署不用求人	部署方式、告警規則、值班表
利害關係人	知道何時能用	早一點的回饋	Sprint Review

「這不是我的問題」出現的地方，就是右欄少了一樣東西的地方。

三種摩擦，都回推到沒寫的東西

缺了什麼	摩擦長這樣	補什麼
例外情境沒寫	「規格上沒寫」對「這不是常識嗎」	Given、When、Then
環境誰負責沒寫	「測試環境又壞了」對「你們自己弄壞的」	環境定義成映像加設定，寫明誰維護
值班與告警沒歸屬	「半夜出事找不到人」對「那不是我的 code」	明確的值班表與告警路由

三種摩擦都能回推到一份沒寫的文件，或一條沒設的規則。

協作不好，是因為對方難搞？

× 大家通常會想

協作不好是因為對方難搞。

換一批人，同一個介面缺口會製造同一種摩擦。

✓ 實際上

「這不是我的問題」通常是流程設計失敗。

跟「把規矩寫進工具」同一個道理：介面要寫成產物，不要靠默契。

QUIZ 04

「這不是我的問題」最常反映什麼？

A 對方態度差

B 介面上少了一樣該交的東西

C 團隊太大

D 工具不夠

想 30 秒，等一下翻下一頁對答案。

QUIZ 04 · ANSWER

「這不是我的問題」最常反映什麼？

☐ A 對方態度差

☒ B 介面上少了一樣該交的東西

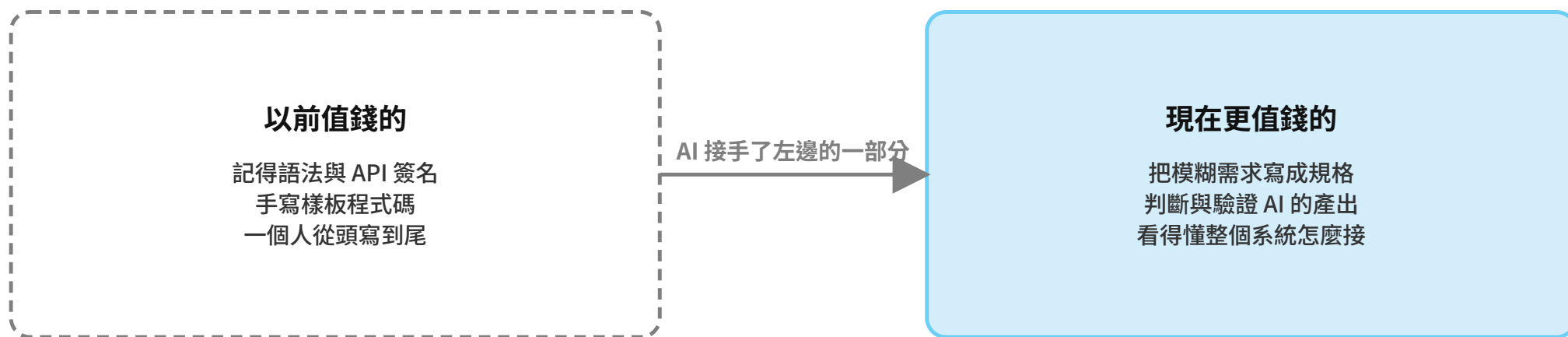
☐ C 團隊太大

☐ D 工具不夠

B。課綱的立場：摩擦多半來自介面沒設計好；那句話出現的地方，就是介面上少了一樣產物的地方。

AI 接手了左邊的一部分

課綱：從純程式碼撰寫者，轉為規格設計者、架構把關者與 AI 協同指揮者



課綱的說法：從純程式碼撰寫者，轉為規格設計者、架構把關者與 AI 協同指揮者

- 1 左邊虛線框是以前值錢的：記語法、手寫樣板、一個人從頭寫到尾
- 2 右邊實線框是現在更值錢的：寫規格、判斷與驗證 AI 的產出、看懂系統怎麼接
- 3 課綱的判斷：判斷、驗證、系統設計更值錢，語法記憶與樣板較不稀缺

三個新角色，前面都練過

新角色	在做什麼	前面哪裡教過
規格設計者	把模糊需求寫成 AI 與人都能執行的規格	驗收條件怎麼寫
架構把關者	判斷 AI 給的做法在你的情境對不對	「架構建議是灰色地帶」
AI 協同指揮者	分清哪些交給 AI，並驗證每一行	AI 家教的三不做與六種問法

三個角色是課綱的分法；右欄的對應是講師的整理。

判斷、驗證、系統設計變得更值錢

變得更值錢	為什麼	變得較不稀缺	為什麼
判斷	AI 給平均答案，合不合你的情境沒人能代勞	語法記憶	查得到、生得出來
驗證	AI 不會卡住，所以你要	樣板程式碼	生成幾秒鐘
系統設計	元件怎麼接、哪裡會壞、錢花在哪	從頭寫到尾的耐力（講師補充）	草稿由 AI 出

項目來自課綱，加一項講師補充；兩欄「為什麼」是講師的解釋。

AI 會寫程式，就不用學程式了？

× 大家通常會想

AI 會寫程式，所以不用學程式了。

語法與樣板這些基本功沒有消失。

✓ 實際上

基本功變成「看得懂才能驗證」的門檻。

AI 讓寫的人變快，沒有讓按 approve 的人變輕；看不懂的人沒有 approve 的資格。

QUIZ 05

「先自己寫驗收條件，再讓 **AI** 挑戰」在練哪個角色？

A 規格設計者

B 架構把關者

C AI 協同指揮者

D 以上都是

想 30 秒，等一下翻下一頁對答案。

QUIZ 05 · ANSWER

「先自己寫驗收條件，再讓 **AI** 挑戰」在練哪個角色？

☐ A 規格設計者

☐ B 架構把關者

☐ C AI 協同指揮者

☒ D 以上都是

D。寫規格是第一個，判斷 AI 的挑戰對不對是第二個，決定順序是第三個。

先自己出手，再讓 **AI** 挑戰

- 一、三個切點：模糊需求、**stacktrace**、進度報告
- 二、**AI** 問的每一條，你也要驗
- 三、能開工的規格，是被問過的規格

三個切點，**同一個順序**

每個框的第一行是你先做的，第二行是 AI 接著做的

三個切點同一個順序：先自己出手，再讓 AI 挑戰

模糊需求

你先列邊界條件
再讓 AI 扮架構師追問

混亂的 stacktrace

你先寫排查假說
再讓 AI 挑戰推論路徑

流水帳進度報告

你先寫三句
再讓 AI 扮敏捷教練修剪

順序反過來就是第 1 單元第 5 課的三不做：讓它寫交付物、接受看不懂的、用它的筆記代替自己的

1 一天裡 AI 進場三個時刻：模糊需求、混亂的 stacktrace、流水帳進度報告

2 三個切點同一個順序：先自己出手，再讓 AI 挑戰

3 順序反過來就是三不做：讓它寫交付物、接受看不懂的、用它的筆記

每個切點：你先做，AI 再問

切點	你先做	AI 接著扮誰	它該問你什麼
模糊需求	寫使用者故事與想得到的驗收條件	嚴苛的資深架構師	邊界條件、例外狀況、沒定義的名詞
混亂的 stacktrace	寫觀察、三個假設與驗證方法	挑戰者	假設為什麼這樣排、驗證能不能區分假設
流水帳進度報告	寫三句：進度、阻礙、下一步	敏捷教練	哪一句還是「我做了什麼」

它問的每一條你都要驗：是**真的沒寫**，還是它沒看到。

先讓 AI 做，拿到的只有成品

拿到什麼	先讓 AI 做	先自己做，再讓它挑戰
規格	看起來完整，不知道它漏了什麼	知道自己漏了什麼，因為是它問出來的
答案	有一個答案，不知道為什麼是它	有三個假設，知道為什麼排這個順序
報告	一段漂亮的報告，講不出裡面的阻礙	阻礙是你寫的，它只是幫你講清楚

左欄拿到的是成品，右欄拿到的是能力；寫過一遍才是你的。

先讓 AI 出草稿再改，比較快？

× 大家通常會想

先讓 AI 出草稿再改，比較快。

改別人的草稿，會順著它的框架想，漏掉的東西你也跟著漏。

✓ 實際上

規格與假設這兩種東西，先自己寫。

先自己寫的那十分鐘，是保住判斷力的十分鐘（講師的看法，不是研究結論）。

“

AI 是挑戰者，
不是代打。

—— 一句話帶走 ——

QUIZ 06

拿到一段模糊需求，第一步是什麼？

- ☐ A 貼給 AI 要它寫規格
- ☐ B 自己先寫使用者故事與驗收條件，再讓 AI 扮架構師追問
- ☐ C 直接開始寫程式
- ☐ D 退回給 PM

想 30 秒，等一下翻下一頁對答案。

QUIZ 06 · ANSWER

拿到一段模糊需求，第一步是什麼？

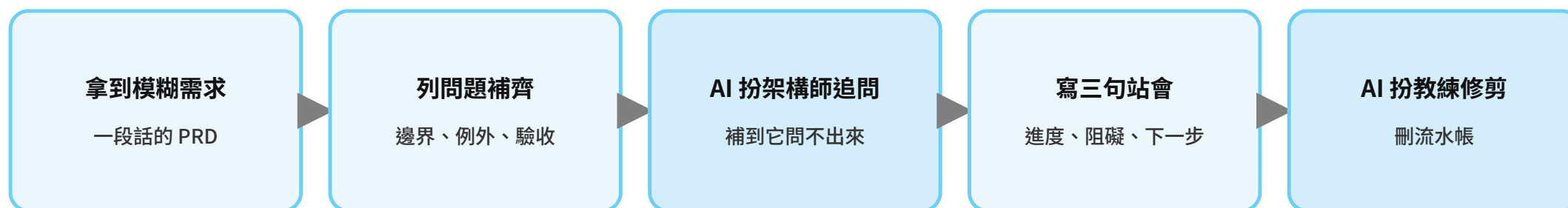
- ☐ A 貼給 AI 要它寫規格
- ☒ B 自己先寫使用者故事與驗收條件，再讓 AI 扮架構師追問
- ☐ C 直接開始寫程式
- ☐ D 退回給 PM

B。三個切點同一個順序：先自己出手，再讓 AI 挑戰；你才知道自己漏了什麼，因為是它問出來的。

零程式碼實作：兩個切點連著做

上色的兩格是 AI 進場的時刻，前面各有一個你先做的框

零程式碼實作：規格補齊與站會表達



交出去的是補齊後的規格、AI 追問紀錄、三句站會的前後版

- 1 五格：拿到模糊需求、列問題補齊、AI 追問、寫三句站會、AI 修剪
- 2 AI 先扮嚴苛的資深架構師，再扮敏捷教練
- 3 交出去三樣：補齊後的規格、AI 追問紀錄、三句站會的前後版

實作材料：一段刻意寫模糊的需求

先讀完，講出「誰、要什麼、為什麼」三個空格哪些沒填

一段話的 PRD

店長要能從後台把訂單匯出來對帳，

最好可以選日期，

格式用 Excel 那種就好，量不會很大。

另外希望之後也能給會計看。

1 可以自己假設、寫在條件旁：筆數上限、檔案格式細節、預設日期範圍

2 必須列成待確認：誰有權限匯出、含不含個資欄位、會計看的是不是同一份

實作：先自己寫，再讓架構師追問

步驟一到三：規格補齊

1

自己寫使用者故事、至少四條 Given When Then（含兩條例外）、三張一天內做完的票

驗收：四條驗收條件可檢查

2

貼給 AI，請它扮架構師

「扮演嚴苛的資深架構師，只用提問引導我補齊邊界條件與例外狀況，不要給答案」

3

來回到它問不出需求範圍內的新問題

補完只多兩三條也照實交；評分看條件可檢查、例外有沒有寫，不看數量

4

它問到需求裡沒有的資訊，分兩類

能合理假設的寫「假設：...」；權限、金流這類列「待確認」交給講師或 PM

實作：三句站會，讓**教練**修剪

步驟四到五：想像這張票做到一半卡住了，用三句話報告

- 1 寫三句站會：進度、阻礙、下一步
- 2 貼給 AI：「扮演敏捷教練，修剪流水帳，強化阻礙與產出進度的表達」
- 3 比對前後版，看阻礙有沒有講得更精準
- 4 繳交：補齊前後的規格、AI 追問紀錄與你的判斷、站會前後版、自審六格

驗收 三句裡有一句是具體的阻礙與需要誰；後版更短，而且阻礙更精準。

三句站會長這樣

流水帳版：「先看了那張票，查了幾個套件，修了一下環境，開了個會，大概做了一半吧」

1 進度：匯出 CSV 的票做到驗收條件第 2 條

第 3 條（空資料提示）還沒開始。

2 阻礙：測試環境的資料庫連不上

需要 SRE 幫我看 DB_URL。

3 下一步：今天中午前把第 3 條做完

下午等連線修好跑驗收。

進度寫「離驗收條件還差哪一條」，阻礙寫到「需要誰做什麼」。

交之前，自審六格

每一格都對回前面教過的一件事

故事

身為誰、想要什麼、為了什麼，三個都有。

條件

每條有 **Given When Then**，含空資料與無權限這類例外。

票

一張一件事，一天內做得完。

追問

AI 問的每一條，你判斷過是真的漏還是它沒看到。

站會

三句，阻礙寫到「需要誰做什麼」。

順序

每一步都是你先寫，AI 後挑戰。

QUIZ 07

站會三句裡，哪一句最常被寫成流水帳？

A 進度

B 阻礙

C 下一步

D 都不會

想 30 秒，等一下翻下一頁對答案。

QUIZ 07 · ANSWER

站會三句裡，哪一句最常被寫成流水帳？

A

進度

B

阻礙

C

下一步

D

都不會

A。「我昨天做了 A、B、C」就是流水帳，該寫「離驗收條件還差哪一條」。

回家作業：六份，一律個人完成

每一份都是先自己做，再讓 AI 挑戰

把站會改寫一次

交前後兩版，加上 AI 扮敏捷教練給的意見。

把一段話拆成票

交使用者故事、四條驗收條件、三張票與 AI 的問題清單。

先寫假說再問 AI

交觀察、三個假設與驗證方法、AI 的挑戰與你的修正。

畫你的介面圖

交四條介面的現況與缺口，加一頁產物草稿。

你的三年地圖

三個角色各寫現況、目標、對應實作，附 AI 的追問與回答。

一天三次

三個切點各交：你先寫的版本、AI 的挑戰、採納與否的理由。

TAKEAWAY

- 01 儀式是回饋點，忘了它解決的問題就走鐘。
- 02 沒寫的例外情境，以後都會變成 bug；Debug 是在證明假設。
- 03 介面要寫成產物；AI 接手了寫，沒接手負責。
- 04 AI 是挑戰者：能開工的規格，是被問過的規格。

Reference

- **Scrum 指南 2020 繁體中文版** [Sprint](#)、[五個儀式](#)、[完成之定義](#)、[跨職能](#)
- **Atlassian：User stories** [使用者故事的格式與驗收條件](#)
- **Cucumber：Gherkin Reference** [Given](#)、[When](#)、[Then](#)
- **Python 文件：Logging HOWTO** [log 的五個等級](#)
- **OpenTelemetry 文件：Traces** [trace](#)、[span](#)、[追蹤 ID](#)
- **課網 D04 程式開發人員的日常** [角色轉變](#)、[三個切點](#)、[實作](#)